1. **Create a Metamask account and share your address.**

Ans: 0x99e16c3c3D92d1c47A96097AEF2E32897Fa9BBbd

2. **Write a one page summary of Ethereum Whitepaper.**

Ans: It starts by describing the history of bitcoin And also describes various other attempts which were made by utilising the idea of Blockchain emerged from Bitcoin .

Firstly it talks about the accounts in Etherium, An Etherium account basically comprises of four fields,

- Nonce : Nonsense is basically like a counter which ensures that each transaction is processed only once and it increases by one after each transaction.
- Ether Balance : It is the current number of ethers present in earn account
- Contract Code: contract code basically refers to any code associated with account,the smart contracts
- Storage: It refers to the account storage which is by default null initially.

There are two types of accounts one is the externally owned account and the other is a contract account:

- **Externally Owned Accounts (EOAs)**: Controlled by private keys, these accounts have no associated code. Transactions from EOAs are created and signed by the account holder.
- **Contract Accounts**: Controlled by their contract code, these accounts execute their code whenever they receive a message or transaction. This allows them to read/write to internal storage and send messages or create contracts.

  Ethereum has its own Ethereum virtual machine where the small contracts are run and thus its maintenance requires fees therefore certain amount of cash is associated for a contract.

Then it talks about the messages and transactions are signed packages.

Transaction: From an EOA it contains receiver address, sender signature ,amount, data , gas and gas price.

Messages : Messages are basically sort of some virtual object sent by contract Which contains data like sender,receiver,amount,data etc.

State Transition:

- **Validation**: Check transaction validity, signature, and nonce.
- **Fee Calculation**: Calculate and subtract transaction fee from sender's balance.
- **Gas Initialization**: Set initial gas and deduct gas per byte.
- **Value Transfer**: Transfer ether from sender to recipient, create account if needed.
- **Code Execution**: Run contract code if recipient is a contract.
- **Revert on Failure**: Revert state changes if execution fails, except for fees.

- **Refund Remaining Gas**: Refund unused gas to sender, pay consumed gas fees to miner.

For Validating a block some amount of work is done.

Let TX be the block's transaction list,S[i] here represents the state,say there are n transactions. For all i in 0...n-1, set S[i+1] = APPLY(S[i],TX[i]). If any applications returns an error, or if the total gas consumed in the block up until this point exceeds the GASLIMIT, return an error.

Let S_FINAL be S[n], but adding the block reward paid to the miner.

3. **Create an ERC-721 contract using OpenZeppelin and mint 3 NFTs to your own address. Verify it by viewing on OpenSea and viewing the transaction info on Sepolia Etherscan. Share your contract address.**

Ans : 0x31113250e9294690407e36123E9Ab197CA1a5D2B

4. **What is a wallet? What are software and hardware wallets? Why don't wallets break the principle of decentralization? List out a few wallets other than Metamask.**

Ans: A cryptocurrency wallet is a tool that allows you to store, send, and receive cryptocurrencies and provide access to crypto assests.

Software Wallets: Software wallets are basically digital applications which helps us to store private keys locally on our device so that we can interact with decentralised applications, send/receive crypto currency etc.

Hardware Wallets: Hardware wallets are basically the physical devices which are designed to store the private keys , within the device there is a chip where the private key is stored securely and can be connected via USB or Bluetooth etc. for use.

Wallets Don't Break Decentralisation:

- Wallets don't store our private key into their servers .They just provide access to our private key using some cryptographic algorithms, also this is the reason why cryptography wallets like metamask ask us to store the recovery password safely, as they also do not have the access to it.
- Cryptographic wallets are also usually open source, so the transparency is maintained, as anyone could see the code behind it.

Some of the examples of software wallets are Meta mask, coinbase wallet, trust wallet and atomic wallet etc.

Some of the examples of hardware wallets are Ledger nano S, Trezor One, keep key etc.

5. **What is the relevance of Zero address? Comment on the effort required to get the private key corresponding to the Zero address.**

Ans:

In Blockchain Zero address is basically like a null address or burn address and does not have a corresponding private key.

For example: Let's say I made a contract where I can post some products and users can buy it so when I initialise products schema/struct containing a buyer field I will initialise the buyer as null address.

Eg: struct Product{

Int id;

String ProductName;

Int price;

Address seller;

Address buyer; --→ can be initialized with zero address initially.

Bool isPurchased;

}

Effort to get corresponding Private Key : As public key can be obtained from a private key in one way that is, given a private key applying some cryptographic hash functions will give the public key. Therefore to come up with the private key corresponding to zero address we have to try all possible private key combinations and check for one which gives zero address.

6. **Find out what is Brave Browser and BAT token**.

Ans: Brave is basically a open source web browser which primarily focuses on maintaining the privacy of users. It also has a intrinsic ad blocking mechanism .Also it does not track the user details for showing privacy preserving ad.

These factors helps brave in significantly improving the browsing speed.

BAT Token: BAT token stands for basic attention token which is a crypto asset of brave browser. Brave does not allow ads but it gives viewers to choose to see privacy preserving ads and in turn the users get a crypto asset called BAT Token(Basic Attention Token), which is as the name suggests, they get a token for their attention.

Users can use this token to support some publishers which they like by sending token or can use this token in many other ways.

7. **Create an ERC-1155 contract using OpenZeppelin and mint NFTs. Understand how it differs from ERC-721.**

Ans: Contract Address: 0x55f0C90c2E0e23Cf18d83ECf672Aa47622C07301

Id: 0,

Link: [A cute Dog - Unidentified contract f3aae1fa-8d40-493e-ad92-822c7e30363c | OpenSea]

Efficiency: ERC-1155 allows batch transfers and operations, making it more gas-efficient compared to ERC-721, which requires separate transactions for each token2.

Smart Contract: ERC-721 requires a new uri for each token, while in ERC-1155 single URI can be shared or customized for specific IDs.

8. **Write a program to generate an Ethereum address. (You can use any library in any language) Note: There is no input to your program.**

Ans:

```
const {
  bufferToHex,
  privateToPublic,
  privateToAddress,
} = require("ethereumjs-util");
const crypto = require("crypto");

function generateEthereumAddress() {
  const privateKey = crypto.randomBytes(32);
  const publicKey = privateToPublic(privateKey);
  const address = privateToAddress(privateKey);

  return {
    privateKey: bufferToHex(privateKey),
    publicKey: bufferToHex(publicKey),
    address: bufferToHex(address),
  };
}

const { address } = generateEthereumAddress();
console.log(address);
```

9. **Write a program to demonstrate the contract address generation.(You can use any library in any language) Note: Sender's address and nonce are the input for your program. Submit your code with some hard coded inputs.**

Ans:

```
const { keccak256, toBuffer, bufferToHex,rlp } = require('ethereumjs-util');
/*
My flow:
```

```
- rlp.encode(concatenated bufferData )
- the result then is hashed by keccak256 cryptographic algorithm
- the last 20 bytes of result in hex formis the contract address.
 */
function generateContractAddress(senderAddress, nonce) {
   const senderAddressBytes = toBuffer(senderAddress)
   const rlpEncoded = rlp.encode([senderAddressBytes,nonce])
   const hash = keccak256(rlpEncoded)

   return bufferToHex(hash.slice(-20))
}
const senderAddress = '0x0643c4555CD0019F5ed126EB47338883288E4A6b';
const nonce = 1;
const contractAddress = generateContractAddress(senderAddress,nonce)
console.log(contractAddress)
```

10. **Create a program to demonstrate transaction signing and verification in Ethereum. Note: Input to your signing function are a transaction (an object having fields like receiver's address, ether to be sent, sender's nonce, gas price, etc..) and a private key of the sender and this function should return a signed transaction. Input to your verification function are the signed transaction and sender's address.**

Ans:

```
const { Wallet, verifyMessage, keccak256 } = require("ethers");
const { rlp } = require("ethereumjs-util");
const {
  bufferToHex,
  privateToPublic,
  privateToAddress,
} = require("ethereumjs-util");
const crypto = require("crypto");

function generateEthereumAddress() {
```

```
  const privateKey = crypto.randomBytes(32);
  const publicKey = privateToPublic(privateKey);
  const address = privateToAddress(privateKey);

  return {
    privateKey: bufferToHex(privateKey),
    publicKey: bufferToHex(publicKey),
    address: bufferToHex(address),
  };
}

async function signTranscation(privateKey, transaction) {
  const params = [
    transaction.nonce,
    transaction.gasPrice,
    transaction.gasLimit,
    transaction.recieverAddress,
    transaction.value,
    transaction.data,
  ];

  // Raw transaction as Buffer
  const rawTransaction = rlp.encode(params);

  const wallet = new Wallet(privateKey);
  const signature = await wallet.signMessage(rawTransaction);
  return signature;
}

function verifyTransaction(transaction, signature, address) {
  const params = [
```

```
    transaction.nonce,
    transaction.gasPrice,
    transaction.gasLimit,
    transaction.recieverAddress,
    transaction.value,
    transaction.data,
  ];

  // Raw transaction as Buffer
  const rawTransaction = rlp.encode(params);
  const signerAddress = verifyMessage(rawTransaction, signature);
  return signerAddress.toLowerCase() == address.toLowerCase();
}

const transaction = {
  nonce: "1",
  gasPrice: "0x4a817c800", // 20000000000 in decimal (20 Gwei)
  gasLimit: "0x5208", // 21000 in decimal
  reciverAddress: "0x98560fd221264e562a756582849eba2307ce1e83",
  value: "0x9184e72a000", // 10000000000000 in decimal (0.01 ETH)
  data: "0x",
};

const { privateKey, address } = generateEthereumAddress();
let sign;
signTranscation(privateKey, transaction)
  .then((signature) => {
    console.log(`Signature: ${signature}`);
    sign = signature;
  })
  .then(() => {
```

```
  const verifySign = sign;
  console.log(verifyTransaction(transaction, verifySign, address));
});
```